

FIT2004 - Algorithms and Data Structures

Seminar 12 - Overview

Rafael Dowsley

Design Principles

Avoid brute force

- Try to avoid brute-force solutions.
- Exhaustive search that tries each possibility (one at a time) in order to determine if it is the solution.
 - ▶ Very often brute-force solutions will lead to algorithms with exponential complexity.
 - ▶ Sometimes brute-force solutions still yield polynomial algorithms. But even in those cases, it is often possible to solve the problem more efficiently using other approaches.

What is this unit about?

- Solving problems with computers - **efficiently**.
- Developing your algorithm toolbox.
- Training your problem solving skills.
- **The unit is not really about programming:**
 - ▶ Python used for assignments, but the subject is really language agnostic.
 - ▶ Algorithms in this unit will be described in English, pseudocode, procedural set of instructions or Python.

Avoid brute force

- Example: given a n -by- n matrix of numbers, find a local maximum (Problem 7 of Week 2 Applied).
 - ▶ $O(n^2)$ brute-force solution is possible by testing for each element if it is a local maximum.
 - ▶ It is possible to solve it in $O(n)$ by using a Divide-and-Conquer approach.

Avoid brute force

- Example: given n points in a plane, find the closest pair of points.
 - ▶ $O(n^2)$ brute-force solution is possible by computing the distance between each pair of points.
 - ▶ It is possible to solve it in $O(n \log n)$ by using a Divide-and-Conquer approach.

Where to start?

- When you have to deal with a problem that is new to you, trying well-know problem solving strategies/algorithm design paradigms is normally a good starting point.

- Is the problem you are trying to solve a variant, extension, special case, ... of another problem for which you already know an efficient algorithm?
- Can you use that efficient algorithm as a subroutine to solve the new problem? A slightly adaption of that algorithm?
- If the new problem is a special case of the more general problem, can you improve the efficiency of the general algorithm for solving this special case?

Reduction/adaptation examples

- The counting inversions problem (Problem 6 of Week 2 Applied) can be easily solved by slightly adapting the Merge Sort algorithm.
- The circulation with demands problem can be reduced to a max-flow problem (and so Ford-Fulkerson method can be used to solve it).

Divide-and-Conquer approach

- Is there an efficient way to divide the problem into smaller sub-instances, solve them and recombine the solutions of the sub-instances to get the solution of the original problem?
- Can be useful when you already know a polynomial time solution but are trying to improve the efficiency further.
- Try to divide the work as equally as possible. Remember the difference in worst-case performance between Merge Sort (which always achieves a good split) and Quicksort with bad pivots.

Greedy algorithms

- Make a locally optimal choice at each stage and commit to it.
- It is often easy to come up with ideas of greedy algorithms to solve a problem, but many times they do not produce optimal solutions.
- **Correctness proofs are very important.**
- Try your greedy ideas on small inputs. If it fails, it will often give you further insights into the problem you are trying to solve. If it works, try to develop a correctness proof for your idea.

- Correctness proofs of greedy algorithms using “exchange arguments”:
 - ▶ Consider an optimal solution and the solution output by your algorithm.
 - ▶ If they are equal, you are done.
 - ▶ Otherwise, keep changing the optimal solution step-by-step so that it becomes more similar to the solution output by your algorithm. Prove that no exchange step makes the solution worse.

- Correctness proofs of greedy algorithms by proving that your solution “stays ahead”:
 - ▶ Prove that your solution is optimal by showing that at each step your solution is ahead or at least equal to every other possible solution (in the relevant metric to your problem).

Greedy algorithms

- Many times you can come up with multiple different greedy ideas, and none of them will work.
- Sometimes you can come up with multiple different greedy ideas, and only one of them works.
- A few times there are multiple different greedy ideas that work (e.g., minimum spanning trees).
- Sometimes greedy is a good heuristic to get a “good” solution to a (combinatorial) problem even if not optimal (and thus used to get “good” solutions to NP-complete problems).

- Do not repeat work, store and re-use!
- Can you show that an optimal solution must be build from optimal solutions to subproblems in a small number of possible ways?
- Can you formulate the recurrence for that?
- **Very powerful algorithm design paradigm!**

- Can your problem be reduced to a max-flow problem? Min-cut problem? Circulation with demands problem?
- Very powerful tool for solving combinatorial problems!
- And, of course, also problems about the flow of material/data.

- Certain data representations are more efficient than others for a given problem.
- Choose the most appropriate data structure for your algorithm as this can impact its efficiency (e.g., priority queues for Dijkstra, union-find for Kruskal, efficient search and retrieval data structures).
- Principle of parsimony: choose the simplest data structure that supports the operations needed by your algorithm (more supported operations generally leads to less efficient data structures).

Where to start?

- Practice is key to improve your intuition about what approaches may be (may not be) fruitful for solving a problem!
- There are a lot of problems and supplementary problems in the Applied Class sheets. Solutions are provided.
- **You will get the most out of it if you try hard to solve them before looking at the solutions.**
- You can look for further problems in algorithms' textbooks.

- These are general guidelines. As always, there are many exceptions. . .

A small change to a problem. . .

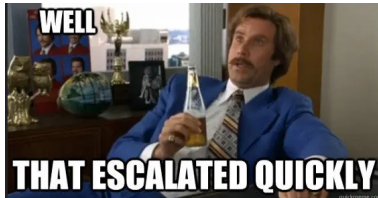
- Sometimes slight changes in the problem definition make new techniques necessary to solve it.
- For example, shortest paths on a graph:
 - ▶ For unweighted edges, it can be solved with an algorithm as simple as BFS.
 - ▶ For edges with non-negative weights only, it is possible to solve it using a greedy algorithm (Dijkstra's algorithm).
 - ▶ For edges with possibly negative weights, it is possible to solve it using Dynamic Programming (Bellman-Ford algorithm).

Another small change to a problem. . .

- Another example, unweighted vs weighted interval scheduling:
 - ▶ Unweighted interval scheduling (Problem 7 of Week 6 Applied): n requests with starting s_i and finishing f_i times. Goal: Find a subset of mutually compatible requests of maximum size.
 - ▶ Greedy solution is possible: (1) sort requests by finishing time, (2) sequentially process the sorted requests adding to the solution each request that is compatible with all requests already in the solution.
 - ▶ Weighted interval scheduling: the requests also have weights w_i . Goal: Find a subset of mutually compatible requests of maximum total weight.
 - ▶ Weighted interval scheduling can be solved using Dynamic Programming.

Yet another small change to a problem...

- Graph colouring:
 - ▶ Deciding whether an undirected graph is 2-colourable (Problem 1 of Week 5 Applied): It can be easily solved using BFS or DFS.
 - ▶ Deciding whether an undirected graph is 3-colourable?
 - ▶ **NP-complete problem!**

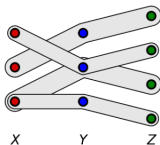


Small tweaks can complicate things a lot

- Sometimes slight changes in the problem definition can make it extremely hard to solve.
- Another example, min-cut vs max-cut:
 - ▶ Ford-Fulkerson algorithm can be used to compute the min-cut in polynomial time.
 - ▶ Computing the max-cut is a NP-hard problem.

Small tweaks can complicate things a lot

- Yet another example, bipartite matching vs 3-dimensional matching:
 - ▶ Bipartite matching can be solved efficiently using network flow techniques as we shown in the Seminar 9.
 - ▶ 3-dimensional matching, the generalisation of bipartite matching in which there are 3 sets X , Y , Z of nodes, and hyperedges that contain one element from each of them is **NP-hard**.



What are the end goals?

- Understanding algorithm design paradigms.
- Developing a gut instinct of algorithmic complexity to be able to make good decisions for real-world deployments.
 - ▶ Internalise the concepts of asymptotical complexity.
 - ▶ Being able to judge wisely when it is critical to analyse in details the complexity of some part of the solution.
- Questions like “Can I do it more efficiently?” should be internalised in your line of thoughts.

Why are algorithms so prominent in interviews?

- They provide a well-defined problem for which the quality of the solutions can be properly measured.
- They allow interviewers to assess your understanding of the CS fundamentals.